



Deep Learning

BITS Pilani

Pilani Campus

Deep Neural Network



Disclaimer and Acknowledgement



- The content for these slides has been obtained from books and various other source on the Internet
- I here by acknowledge all the contributors for their material and inputs.
- I have provided source information wherever necessary
- I have added and modified the content to suit the requirements of the course

Session Agenda



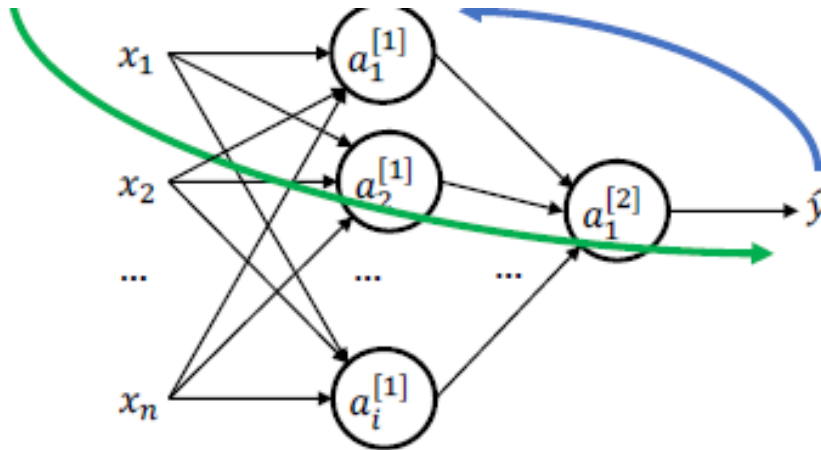
- Back Propagation

Training: Go Forwards, then Backwards...



**Step 3: Calculate partial derivatives
(using Backpropagation)**

**Step 1: Calculate $\hat{y}$ using
computation graph.**



Step 2: Determine the cost.

**Step 4: Update *each*
parameter**

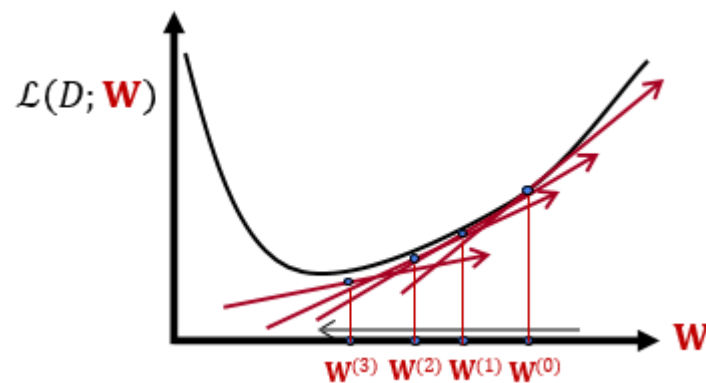
$$J = -\frac{1}{m} \left(\sum_{i=1}^m y^i \log(\hat{y}^{(i)}) + \sum_{i=1}^m (1 - y^{(i)}) \log(1 - \hat{y}^{(i)}) \right)$$

$$w = w - \alpha \frac{\partial J}{\partial w}; b = b - \alpha \frac{\partial J}{\partial b}$$

Training Neural Networks: optimizing parameters



- We are given an architecture though its weights $\mathbf{W}$.
- Also given a training data $D = \{(\mathbf{x}_i, y_i)\}$
- We are given a loss function $\mathcal{L}(D; \mathbf{W})$
- We can use **gradient descent** to minimize the loss.
- At each step, the **weight vector** is modified in the **direction that produces the steepest descent** along the error surface.



Algorithm for Gradient Computation – Backpropagation



- For each sub-parameter $W_i \in \mathbf{W}$: $w_i^{t+1} = w_i^t - \alpha \frac{\partial \mathcal{L}}{\partial w_i}$

It all comes down to effectively computing $\frac{\partial \mathcal{L}}{\partial w_i}$

- How to **efficiently** compute $\frac{\partial \mathcal{L}}{\partial w_i}$ for all parameters?
- The calculus just gets a bit more complicated for a neural network
- **Depth** gives more representational capacity to neural networks.
- However, training **deep** nets is **not trivial**.
- The solution is “Backpropagation” algorithm!
- Backpropagation is a systematic **numerical** method to calculate the partial derivatives (i.e. partial derivative of the cost w.r.t. each parameter)

Key Intuitions Required for Backpropagation

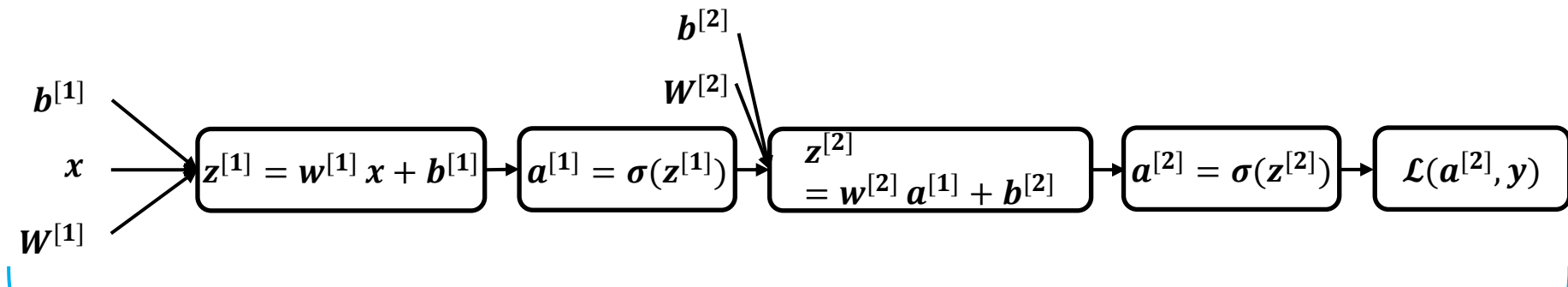


1. Gradient Descent
 - Change the weights $\mathbf{W}$ in the direction of gradient to minimize the error function.
2. Chain Rule
 - Use the chain rule to calculate the weights of the intermediate weights
3. Dynamic Programming (Memorization)
 - Parameter gradients depend on the gradients of the earlier layers!
 - So, when computing gradients at each layer, we can reuse gradients computed for higher layers for lower layers (i.e., memorization).

The Computation Graph of A Neural Network



- we can represent any neural network in terms of a computation graph
- The loss function can be computed by moving from left to right

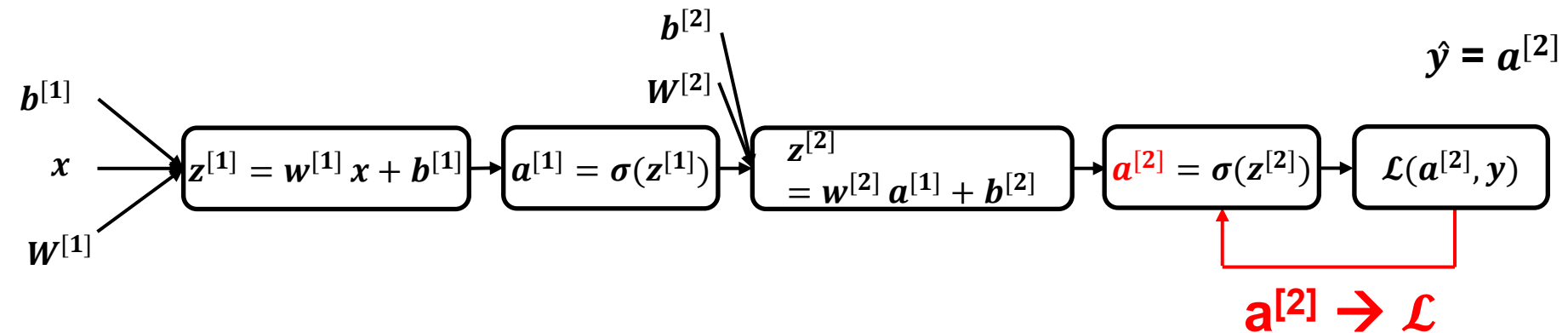


The corresponding computation graph

Cross entropy cost function:

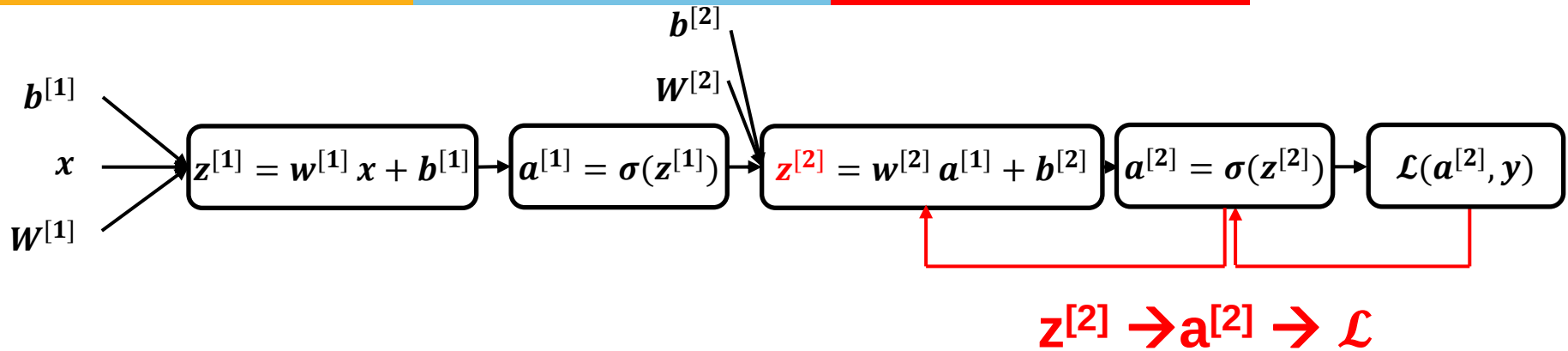
$$\text{Cost}(\hat{y}, y) = -y * \log \hat{y} - (1 - y) \log(1 - \hat{y})$$

Backward Propagation



$$\begin{aligned}\frac{\partial \mathcal{L}}{\partial a^{[2]}} &= \frac{\partial}{\partial a^{[2]}} (-y \log(a^{[2]}) - (1 - y) \log(1 - a^{[2]})) \\ &= \frac{-y}{a^{[2]}} + \frac{(1 - y)}{(1 - a^{[2]})}\end{aligned}$$

Backward Propagation

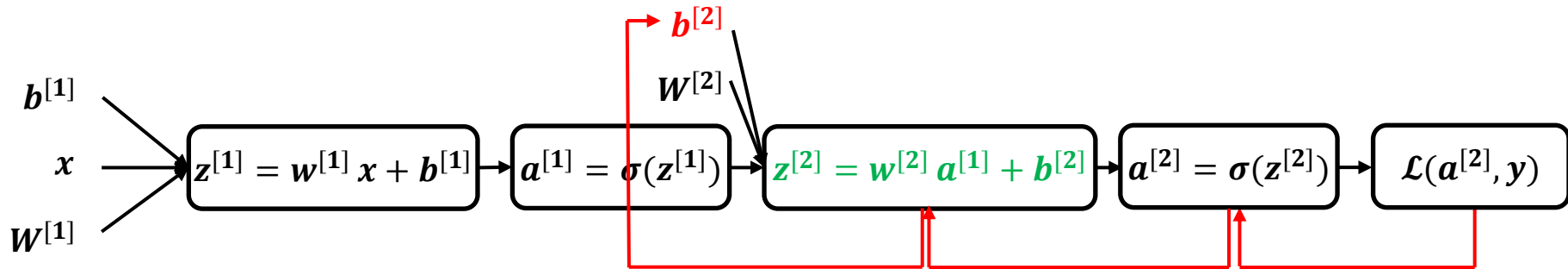


$$a^{[2]} = \sigma(z^{[2]})$$

$$\frac{\partial a^{[2]}}{\partial z^{[2]}} = a^{[2]}(1 - a^{[2]})$$

$$\frac{\partial \mathcal{L}}{\partial z^{[2]}} = \frac{\partial \mathcal{L}}{\partial a^{[2]}} \times \frac{\partial a^{[2]}}{\partial z^{[2]}} = a^{[2]} - y$$

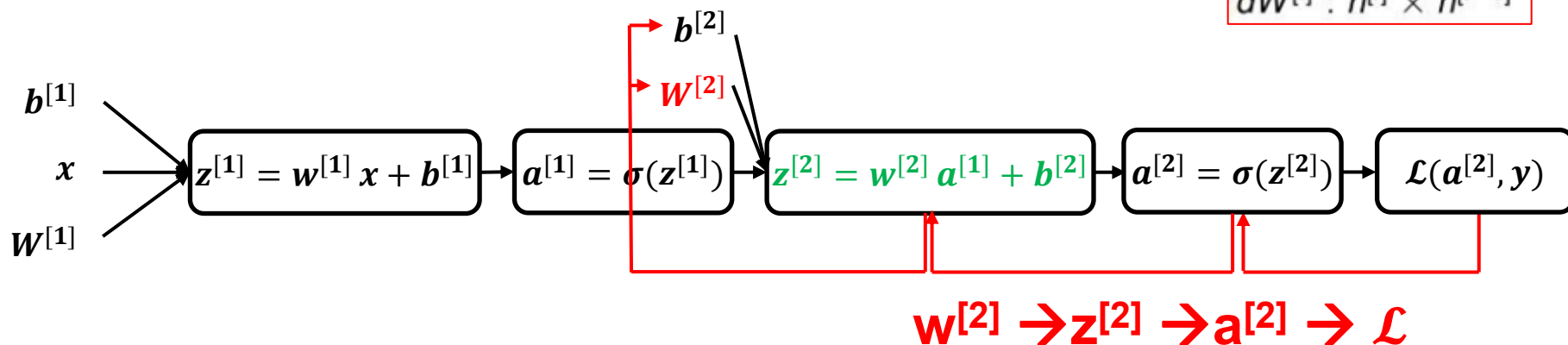
Backward Propagation



$$\frac{\partial \mathcal{L}}{\partial b^{[2]}} = \left(\frac{\partial \mathcal{L}}{\partial a^{[2]}} \times \frac{\partial a^{[2]}}{\partial z^{[2]}} \right) \times \frac{\partial z^{[2]}}{\partial b^{[2]}} = \frac{\partial \mathcal{L}}{\partial z^{[2]}}$$

The diagram shows the chain rule calculation for the gradient of the loss with respect to the bias $b^{[2]}$. The first term, $\frac{\partial \mathcal{L}}{\partial a^{[2]}} \times \frac{\partial a^{[2]}}{\partial z^{[2]}}$, is circled in blue and labeled with $\frac{\partial \mathcal{L}}{\partial z^{[2]}}$. The second term, $\frac{\partial z^{[2]}}{\partial b^{[2]}}$, is also circled in blue and labeled with 1.

Backward Propagation



$$dz^{[l]} : n^{[l]} \times 1$$

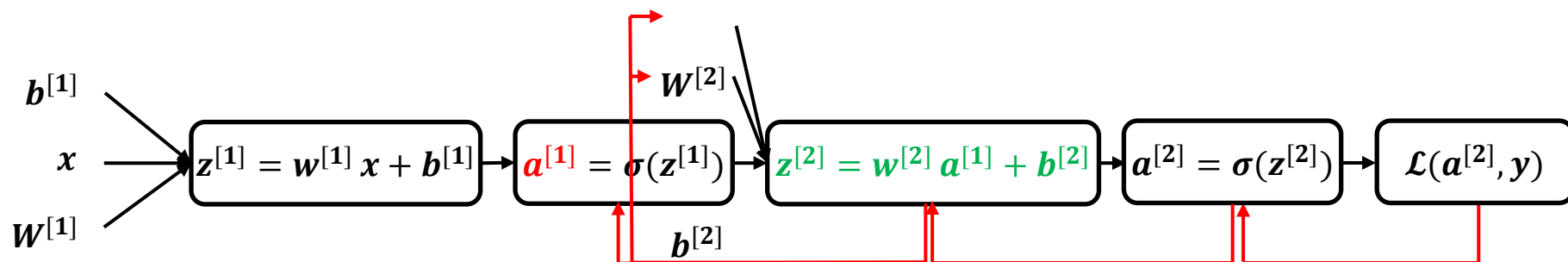
$$a^{[l]} : n^{[l]} \times 1$$

$$dW^{[l]} : n^{[l]} \times n^{[l-1]}$$

$$\frac{\partial \mathcal{L}}{\partial W^{[2]}} = \frac{\partial \mathcal{L}}{\partial a^{[2]}} \times \frac{\partial a^{[2]}}{\partial z^{[2]}} \times \frac{\partial z^{[2]}}{\partial W^{[2]}} = \frac{\partial \mathcal{L}}{\partial z^{[2]}} \times a^{[1]T}$$

Dimension of $dz^{[2]}$ is $(n^{[2]}, 1)$
 Dimension of $a^{[1]}$ is $(n^{[1]}, 1)$
 Hence transpose of $a^{[1]}$
 Dimension of $dW^{[2]}$ is $(n^{[2]}, n^{[1]})$

Backward Propagation



$a^{[1]} \rightarrow z^{[2]} \rightarrow a^{[2]} \rightarrow \mathcal{L}$

$$\frac{\partial \mathcal{L}}{\partial a^{[1]}} = \left(\frac{\partial \mathcal{L}}{\partial a^{[2]}} \times \frac{\partial a^{[2]}}{\partial z^{[2]}} \right) \times \frac{\partial z^{[2]}}{\partial a^{[1]}} = \mathbf{W}^{[2]T} \times \frac{\partial \mathcal{L}}{\partial z^{[2]}}$$

$\frac{\partial \mathcal{L}}{\partial z^{[2]}}$

Dimension of $dz^{[2]}$ is $(n^{[2]}, 1)$
Dimension of $W^{[2]}$ is $(n^{[2]}, n^{[1]})$
Dimension of $da^{[1]}$ is $(n^{[1]}, 1)$

But, the calculation of $da^{[1]}$ is not required.

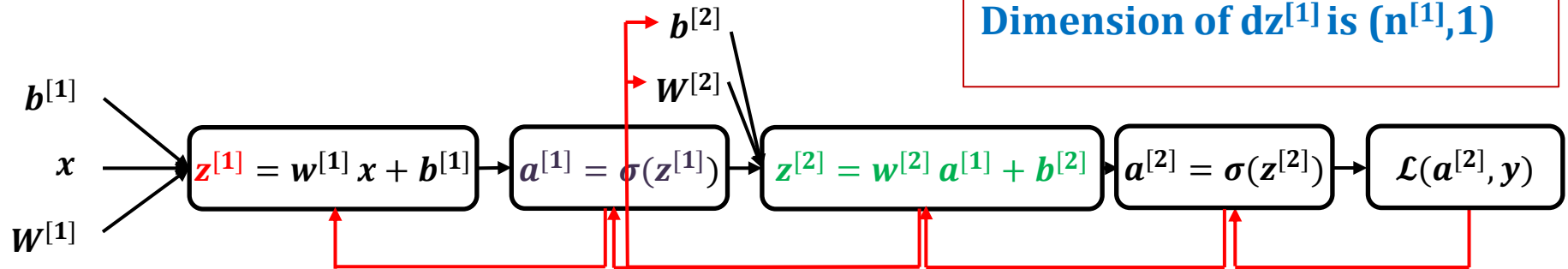
Backward Propagation

innovate

achieve

lead

Dimension of $dz^{[2]}$ is $(n^{[2]}, 1)$
 Dimension of $W^{[2]}$ is $(n^{[2]}, n^{[1]})$
 Dimension of $da^{[1]}$ is $(n^{[1]}, 1)$
 Dimension of $dz^{[1]}$ is $(n^{[1]}, 1)$



$z^{[1]} \rightarrow a^{[1]} \rightarrow z^{[2]} \rightarrow a^{[2]} \rightarrow \mathcal{L}$

$$\frac{\partial \mathcal{L}}{\partial z^{[1]}} = \frac{\partial \mathcal{L}}{\partial a^{[2]}} \times \frac{\partial a^{[2]}}{\partial z^{[2]}} \times \frac{\partial z^{[2]}}{\partial a^{[1]}} \times \frac{\partial a^{[1]}}{\partial z^{[1]}} = \left(w^{[2]T} \times \frac{\partial \mathcal{L}}{\partial z^{[2]}} \right) * a^{[1]}(1 - a^{[1]})$$

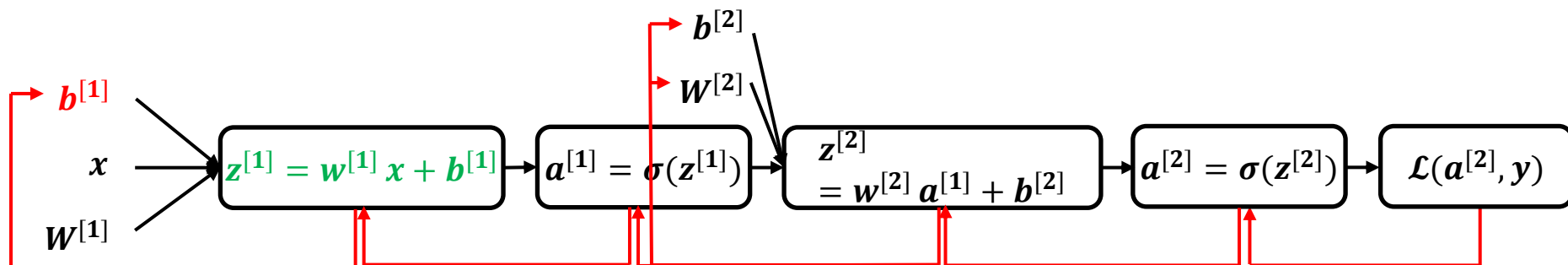
Element-wise product

$$\frac{\partial \mathcal{L}}{\partial a^{[1]}}$$

$$a^{[1]} = \sigma(z^{[1]})$$

$$\frac{\partial a^{[1]}}{\partial z^{[1]}} = a^{[1]}(1 - a^{[1]})$$

Backward Propagation

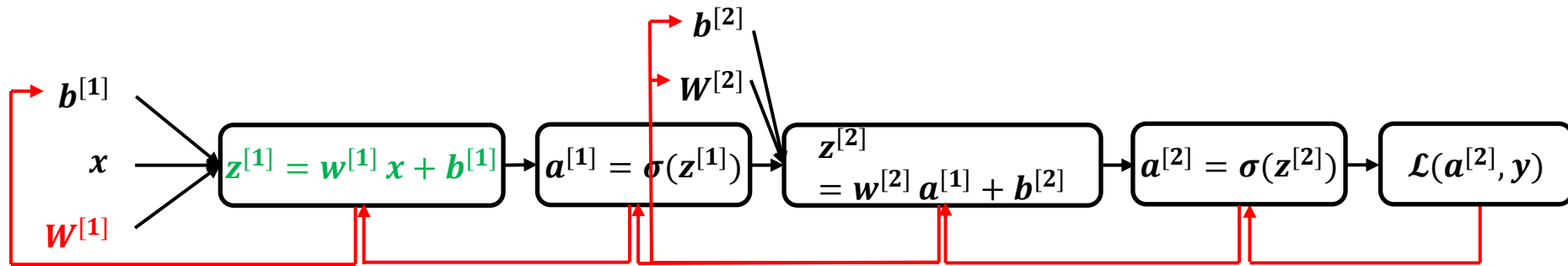


$$b^{[1]} \rightarrow z^{[1]} \rightarrow a^{[1]} \rightarrow z^{[2]} \rightarrow a^{[2]} \rightarrow \mathcal{L}$$

$$\frac{\partial \mathcal{L}}{\partial b^{[1]}} = \frac{\partial \mathcal{L}}{\partial a^{[2]}} \times \frac{\partial a^{[2]}}{\partial z^{[2]}} \times \frac{\partial z^{[2]}}{\partial a^{[1]}} \times \frac{\partial a^{[1]}}{\partial z^{[1]}} \times \frac{\partial z^{[1]}}{\partial b^{[1]}} = \frac{\partial \mathcal{L}}{\partial z^{[1]}}$$

1

Backward Propagation



$$W^{[1]} \rightarrow z^{[1]} \rightarrow a^{[1]} \rightarrow z^{[2]} \rightarrow a^{[2]} \rightarrow \mathcal{L}$$

$$\frac{\partial \mathcal{L}}{\partial w^{[1]}} = \frac{\partial \mathcal{L}}{\partial a^{[2]}} \times \frac{\partial a^{[2]}}{\partial z^{[2]}} \times \frac{\partial z^{[2]}}{\partial a^{[1]}} \times \frac{\partial a^{[1]}}{\partial z^{[1]}} \times \frac{\partial z^{[1]}}{\partial w^{[1]}} = \frac{\partial \mathcal{L}}{\partial z^{[1]}} x^T$$

Backward Propagation: Summary



One training example

For all training example

For cross entropy loss and sigmoid activation in the last layer

$$dz^{[2]} = \frac{\partial \mathcal{L}}{\partial z^{[2]}} = a^{[2]} - y$$

$$db^{[2]} = \frac{\partial \mathcal{L}}{\partial b^{[2]}} = dz^{[2]}$$

$$dw^{[2]} = \frac{\partial \mathcal{L}}{\partial W^{[2]}} = dz^{[2]} \cdot a^{[1]T}$$

$$dz^{[1]} = \frac{\partial \mathcal{L}}{\partial z^{[1]}} = (w^{[2]T} dz^{[2]}) * g'(z^{[1]})$$

$$db^{[1]} = \frac{\partial \mathcal{L}}{\partial b^{[1]}} = dz^{[1]}$$

$$dw^{[1]} = \frac{\partial \mathcal{L}}{\partial W^{[1]}} = dz^{[1]} \cdot x^T$$

$$dZ^{[2]} = \frac{\partial \mathcal{L}}{\partial Z^{[2]}} = A^{[2]} - Y$$

$$db^{[2]} = \frac{\partial \mathcal{L}}{\partial b^{[2]}} = \frac{1}{m} \sum dz^{[2]}$$

$$dW^{[2]} = \frac{\partial \mathcal{L}}{\partial W^{[2]}} = \frac{1}{m} (dz^{[2]} \cdot A^{[1]T})$$

$$dZ^{[1]} = \frac{\partial \mathcal{L}}{\partial Z^{[1]}} = (W^{[2]T} dz^{[2]}) * g'(Z^{[1]})$$

$$db^{[1]} = \frac{\partial \mathcal{L}}{\partial b^{[1]}} = \frac{1}{m} \sum dz^{[1]}$$

$$dW^{[1]} = \frac{\partial \mathcal{L}}{\partial W^{[1]}} = \frac{1}{m} (dz^{[1]} \cdot X^T)$$



Equations for layer l

One training example

Input $a^{[l]}$

Output $da^{[l-1]}, dw^{[l]}, db^{[l]}$

$$dz^{[l]} = da^{[l]} * g^{[l]'}(z^{[l]})$$

$$db^{[l]} = dz^{[l]}$$

$$dW^{[l]} = dz^{[l]} \cdot a^{[l-1]}$$

$$da^{[l-1]} = W^{[l]T} dz^{[l]}$$

For all training example

$$dZ^{[l]} = dA^{[l]} * g^{[l]'}(Z^{[l]})$$

$$db^{[l]} = \frac{1}{m} \sum dz^{[l]}$$

$$dW^{[l]} = \frac{1}{m} (dZ^{[l]} \cdot A^{[l-1]T})$$

$$dA^{[l-1]} = W^{[l]T} dZ^{[l]}$$



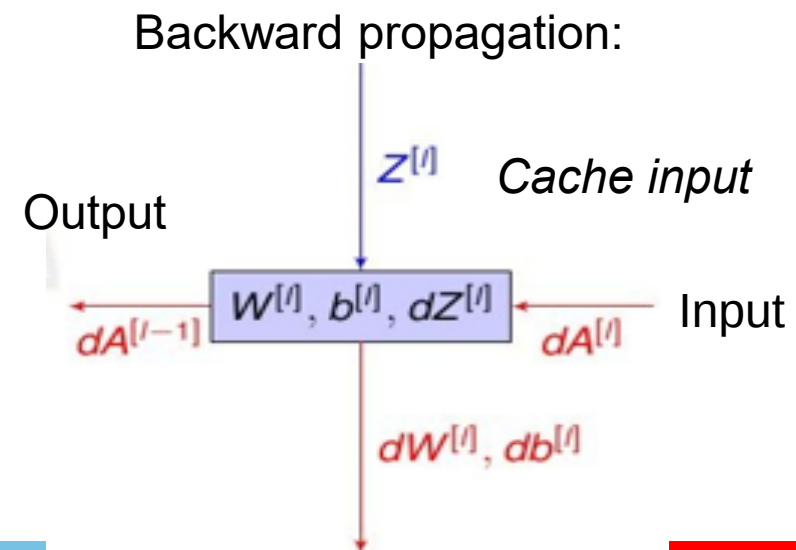
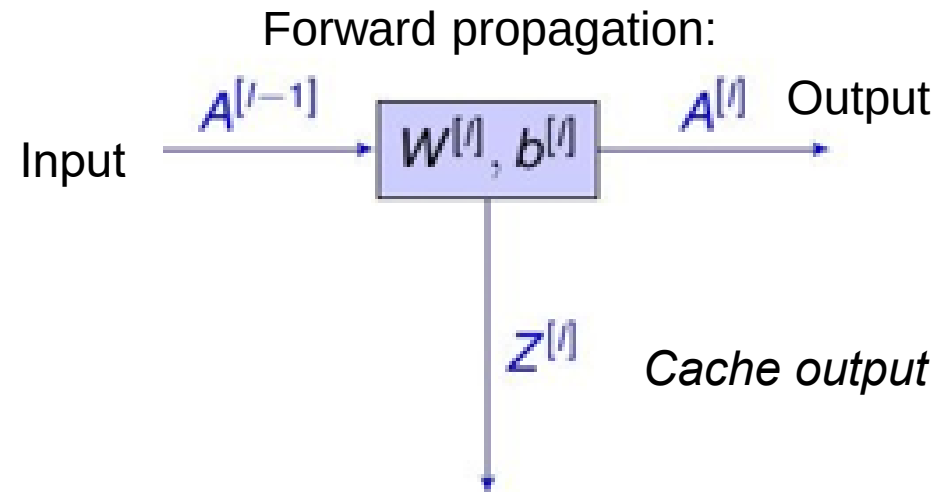
Scaling up for L layers and all training examples in NN

Algorithm 4: Forward Propagation

```
1 for l in range (1, L) do
2   |  $Z^{[l]} = W^{[l]} A^{[l-1]} + b^{[l]}$ 
3   |  $A^{[l]} = g^{[l]} (Z^{[l]})$ 
4  $\hat{y} = A^{[L]}$ 
5  $J = L(\hat{y}, y) + \lambda \Omega(\theta)$ 
```

Algorithm 5: Backward Propagation

```
1  $da^{[L]} = \frac{-y}{a} + \frac{1-y}{1-a}$ 
2 for l in range (1, L) do
3   |  $dZ^{[l]} = dA^{[l]} * g'^{[l]} (Z^{[l]})$ 
4   |  $dW^{[l]} = dZ^{[l]} \cdot A^{[l-1]T}$ 
5   |  $db^{[l]} = dZ^{[l]}$ 
6   |  $dA^{[l-1]} = W^{[l]T} \cdot dZ^{[l]}$ 
```



Scaling up for L layers and all training examples in NN

Algorithm 6: Weight Updation

```
1 for  $l$  in range  $(1, L)$  do  
2    $W^{[l]} = W^{[l]} - \text{learning rate} * dW^{[l]}$   
3    $b^{[l]} = b^{[l]} - \text{learning rate} * db^{[l]}$ 
```

Scaling up for L layers and all training examples in NN

Notations

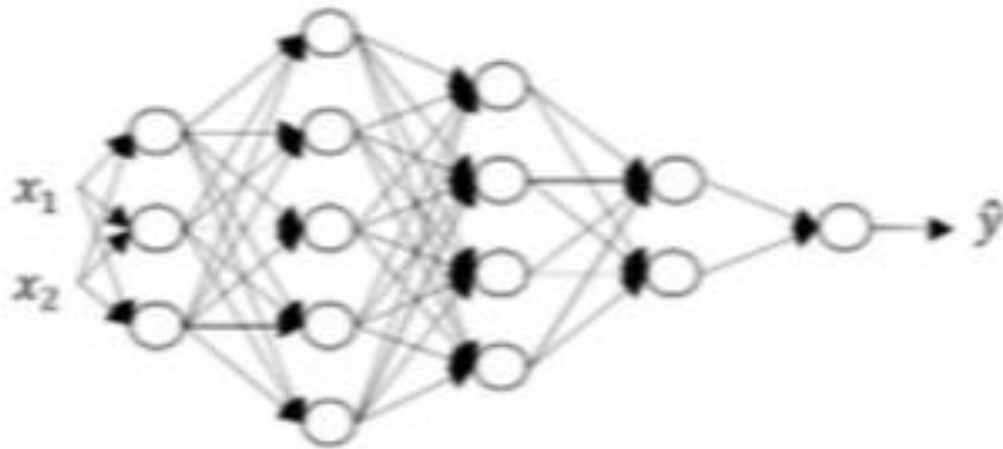
- L : Number of layers
- $n^{[l]}$: Number of units in layer l
- $w^{[l]}$: Weights for layer l
- $b^{[l]}$: Bias for layer l
- $z^{[l]}$: Hypothesis for layer l
- $g^{[l]}$: Activation Function used for layer l
- $a^{[l]}$: Activation for layer l

Matrix Dimensions

- $W^{[l]} : n^{[l]} \times n^{[l-1]}$
- $b^{[l]} : n^{[l]} \times 1$
- $z^{[l]} : n^{[l]} \times 1$
- $a^{[l]} : n^{[l]} \times 1$
- $dW^{[l]} : n^{[l]} \times n^{[l-1]}$
- $db^{[l]} : n^{[l]} \times 1$
- $dz^{[l]} : n^{[l]} \times 1$
- $da^{[l]} : n^{[l]} \times 1$

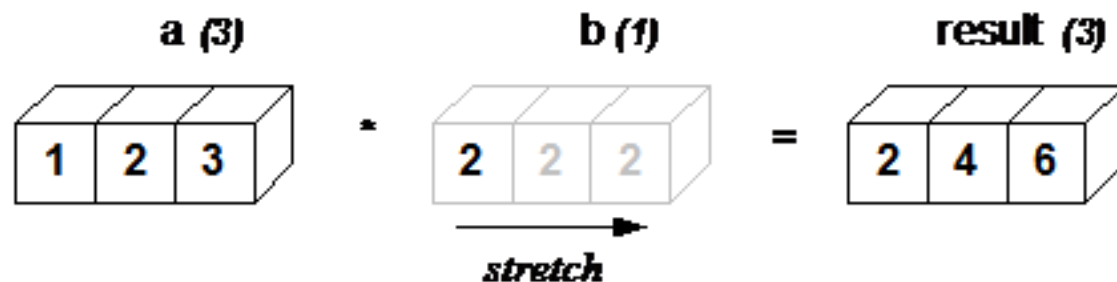
Example

Calculate all matrix dimensions and total number of parameters

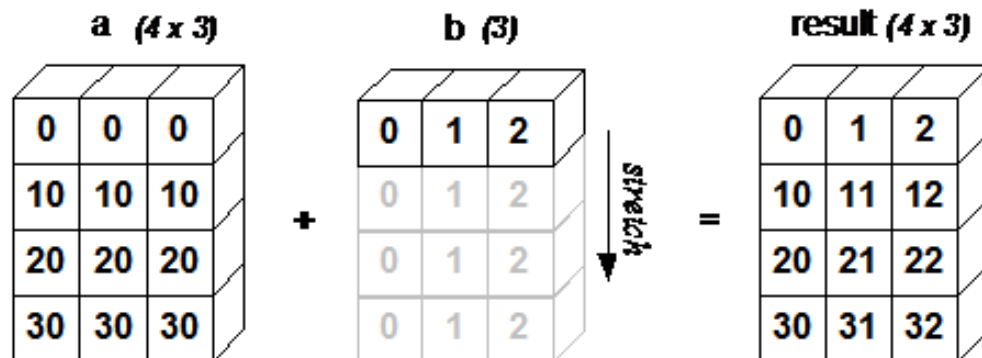


- $W^{[l]} : n^{[l]} \times n^{[l-1]}$
- $b^{[l]} : n^{[l]} \times 1$
- $z^{[l]} : n^{[l]} \times 1$
- $a^{[l]} : n^{[l]} \times 1$
- $dW^{[l]} : n^{[l]} \times n^{[l-1]}$
- $db^{[l]} : n^{[l]} \times 1$
- $dz^{[l]} : n^{[l]} \times 1$
- $da^{[l]} : n^{[l]} \times 1$

Array broadcasting

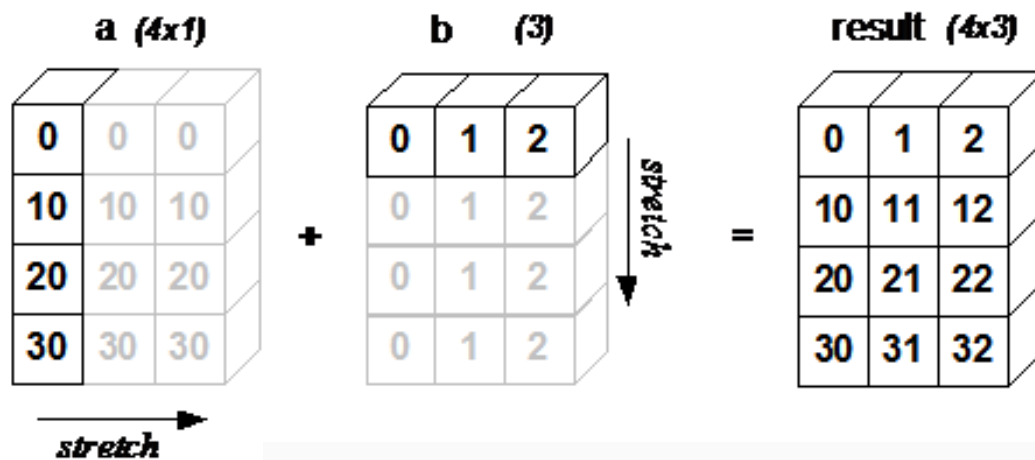


```
>>> from numpy import array
>>> a = array([1.0,2.0,3.0])
>>> b = 2.0
>>> a * b
array([ 2.,  4.,  6.])
```



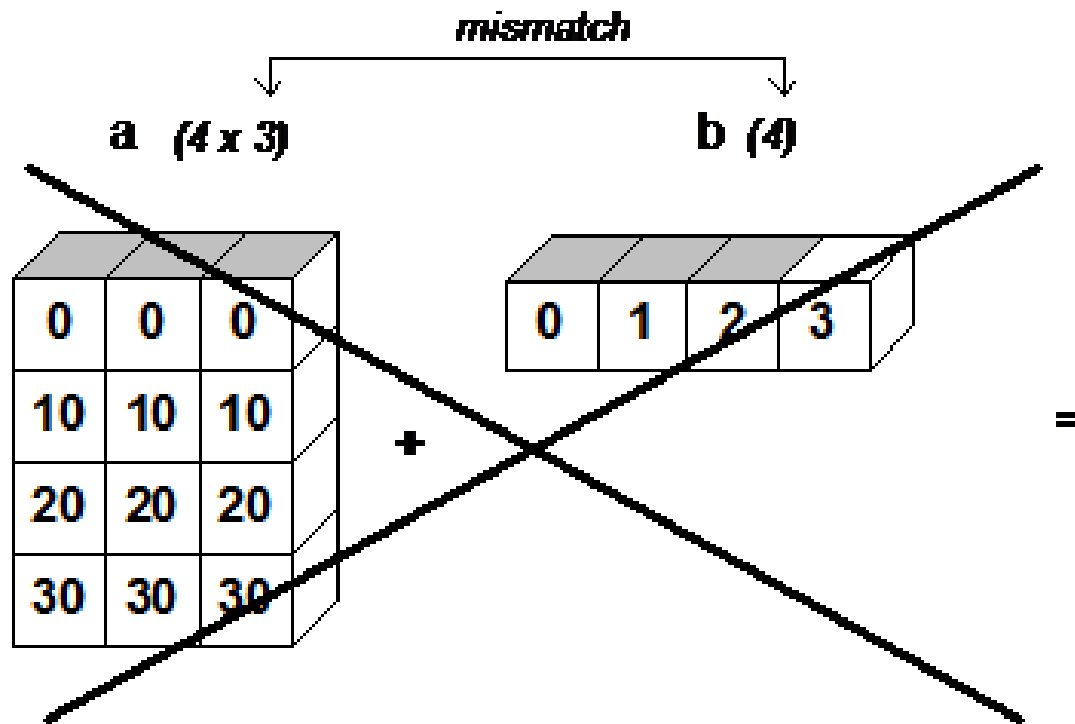
```
>>> from numpy import array
>>> a = array([[ 0.0,  0.0,  0.0],
...           [10.0, 10.0, 10.0],
...           [20.0, 20.0, 20.0],
...           [30.0, 30.0, 30.0]])
>>> b = array([1.0, 2.0, 3.0])
>>> a + b
array([[ 1.,  2.,  3.],
       [11., 12., 13.],
       [21., 22., 23.],
       [31., 32., 33.]])
```

Array broadcasting



```
>>> from numpy import array, newaxis
>>> a = array([0.0, 10.0, 20.0, 30.0])
>>> b = array([1.0, 2.0, 3.0])
>>> a[:,newaxis] + b
array([[ 1.,  2.,  3.],
       [11., 12., 13.],
       [21., 22., 23.],
       [31., 32., 33.]])
```

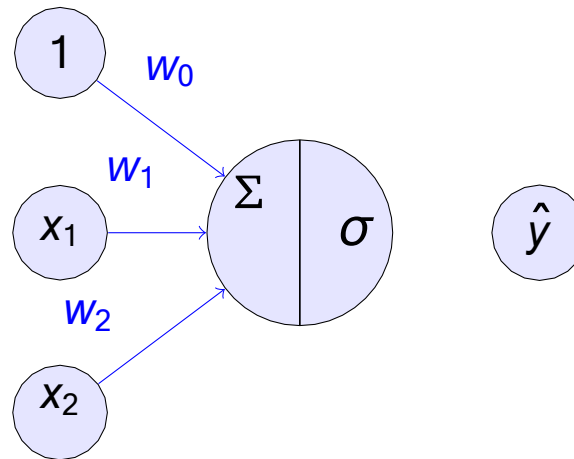

Array broadcasting



Exercise - MSE Loss



Consider the neural network with two inputs x_1 and x_2 and the initial weights are $w_0 = 0.5$, $w_1 = 0.8$, $w_2 = 0.3$. Draw the network, compute the output, mean squared loss function and weight updation when the input is $(1, 0)$, the learning rate is 0.01 and target output is 1. Assume any other relevant information.



Solution



$$\hat{y} = \sigma(w_0 + w_1x_1 + w_2x_2) = \sigma(0.5 + 0.8 * 1 + 0.3 * 0) = \sigma(1.3) = 0.7858$$

$$MSE = \frac{1}{2m} \sum_{i=1}^m (y - \hat{y})^2 = \frac{1}{2} (1 - 0.7858)^2 = \frac{0.04588}{2} = 0.02294$$

$$\mathbf{g} = \frac{2}{2m} \sum_{i=1}^m (y - \hat{y}) = (1 - 0.7858) = 0.2142$$

$$new\ w \leftarrow old\ w - \eta * \mathbf{g}$$

$$w_0 = 0.5 - 0.01 * 0.2142 = 0.4978$$

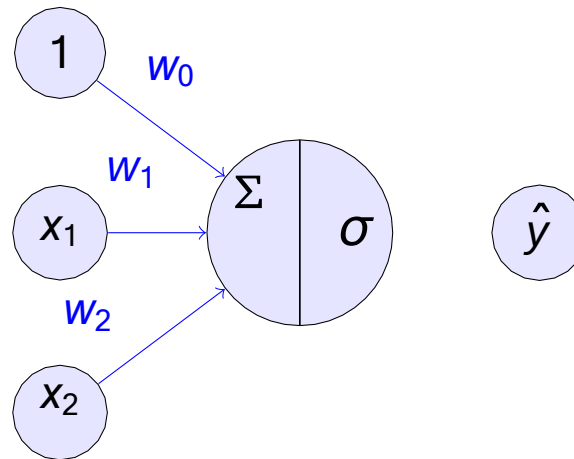
$$w_1 = 0.8 - 0.01 * 0.2142 = 0.7978$$

$$w_2 = 0.3 - 0.01 * 0.2142 = 0.2978$$

EXERCISE - BCE



Consider the neural network with two inputs x_1 and x_2 and the initial weights are $w_0 = 0.5$, $w_1 = 0.8$, $w_2 = 0.3$. Draw the network, compute the output, binary cross entropy loss function and weight updation when the input is $(1, 0)$, the learning rate is 0.01 and target output is 1. Assume any other relevant information.



Solution



$$\hat{y} = \sigma(w_0 + w_1x_1 + w_2x_2) = \sigma(0.5 + 0.8 * 1 + 0.3 * 0) = \sigma(1.3) = 0.7858$$

$$\begin{aligned} Loss &= \frac{1}{m} \sum_{i=1}^m - [y^{(i)} \log \hat{y}^{(i)} + (1 - y^{(i)}) \log(1 - \hat{y}^{(i)})] \\ &= -[1 \ln 0.7858 + (1 - 1) \ln(1 - 0.7858)] = 0.24 \end{aligned}$$

$$\mathbf{g} = \sum_{i=1}^m (y - \hat{y})z = (1 - 0.7858) * 1.3 = 0.278$$

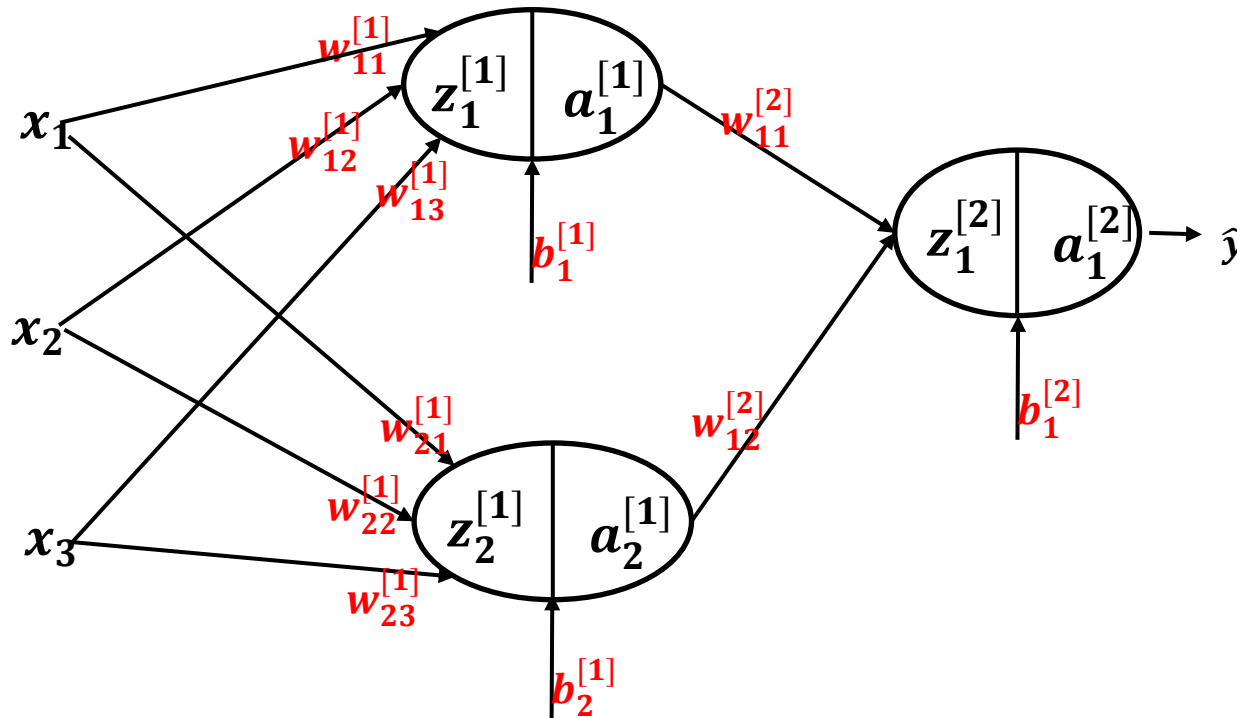
$$W \leftarrow W - \eta * \mathbf{g}$$

$$w_0 = 0.5 - 0.01 * 0.278 = 0.497$$

$$w_1 = 0.8 - 0.01 * 0.278 = 0.797$$

$$w_2 = 0.3 - 0.01 * 0.278 = 0.297$$

Exercise (without vectorization)



$$\begin{aligned}W_{11}^{[1]} &= 0.2 \\W_{12}^{[1]} &= 0.4 \\W_{13}^{[1]} &= -0.5 \\W_{21}^{[1]} &= -0.3 \\W_{22}^{[1]} &= 0.1 \\W_{23}^{[1]} &= 0.2\end{aligned}$$

$$\begin{aligned}W_{11}^{[2]} &= -0.3 \\W_{12}^{[2]} &= -0.2\end{aligned}$$

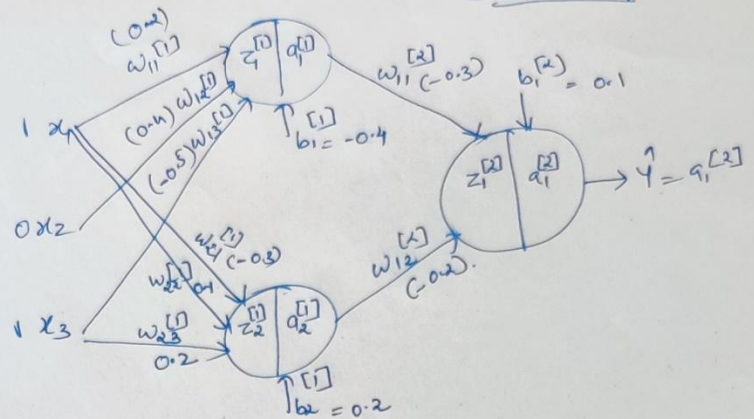
$$\begin{aligned}b_1^{[1]} &= -0.4 \\b_2^{[1]} &= -0.2 \\b_1^{[2]} &= 0.1\end{aligned}$$

$$\alpha = 0.9$$

For $X = \{1, 0, 1\}$ and $y = 1$

Find the cross entropy loss and weight updates after 1st iteration

Forward propagation



$$\begin{aligned} \textcircled{1} \quad z_1^{[1]} &= w_{11}^{[1]} \cdot x_1 + w_{12}^{[1]} \cdot x_2 + w_{13}^{[1]} \cdot x_3 + b_1^{[1]} \\ &= 0.2 \times 1 + 0.4 \times 0 + (-0.5) \times 1 + (-0.4) = \boxed{-0.7} \\ a_1^{[1]} &= \frac{1}{1 + e^{-(-0.7)}} = \boxed{0.331} \end{aligned}$$

$$\begin{aligned} \textcircled{2} \quad z_2^{[1]} &= w_{21}^{[1]} \cdot x_1 + w_{22}^{[1]} \cdot x_2 + w_{23}^{[1]} \cdot x_3 + b_2^{[1]} \\ &= (-0.3) \times 1 + (0.1) \times 0 + (0.2) \times 1 + 0.2 = \boxed{0.1} \\ a_2^{[1]} &= \frac{1}{1 + e^{-0.1}} = \boxed{0.525} \end{aligned}$$

$$\begin{aligned} \textcircled{3} \quad z_1^{[2]} &= w_{11}^{[2]} \cdot a_1^{[1]} + w_{12}^{[2]} \cdot a_2^{[1]} + b_1^{[2]} \\ &= (-0.3 \times 0.331) + (-0.2 \times 0.525) + 0.1 = \boxed{-0.104} \\ a_1^{[2]} &= \frac{1}{1 + e^{-(-0.104)}} = \boxed{0.474} \end{aligned}$$

$$\boxed{\hat{y} = 0.474, y = 1}$$

$$\begin{aligned} \text{CE loss} &= y \cdot \log_2 \hat{y} + (1-y) \cdot \log_2 (1-\hat{y}) \\ &= 1 \cdot \log_2 0.474 + (1-1) \cdot \log_2 (1-0.474) \\ &= \boxed{-1.07738} \end{aligned}$$

$$\begin{aligned} \text{MSE} &= \frac{1}{2} (y - \hat{y})^2 \\ &= \frac{1}{2} (1 - 0.474)^2 \\ &= \boxed{0.138396} \end{aligned}$$

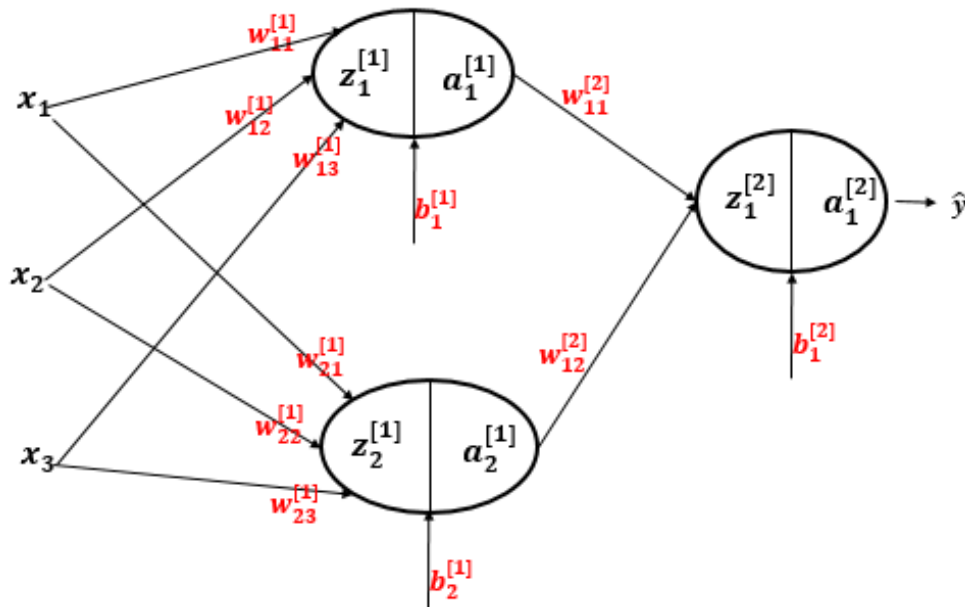
innovate

achieve

lead

Continued.....

For the layer 2



$$dz_1^{[2]} = \frac{\partial \mathcal{L}}{\partial z_1^{[2]}} = a_1^{[2]} - y \quad \leftarrow \text{For Cross entropy cost function}$$

$$dz_1^{[2]} = \frac{\partial \mathcal{L}}{\partial z_1^{[2]}} = a_1^{[2]} (1 - a_1^{[2]}) (y - a_1^{[2]}) \quad \leftarrow \text{For MSE cost function}$$

olp node - Layer-2

$$1) dz_1^{[2]} = a_1^{[2]} - y = 0.474 - 1 = -0.526.$$

$$2) dw_{11}^{[2]} = dz_1^{[2]} \cdot a_1^{[1]} = -0.526 \times 0.331 = -0.174106.$$

$$3) dw_{12}^{[2]} = dz_1^{[2]} \cdot a_2^{[1]} = -0.526 \times 0.525 = -0.27615$$

$$4) db_1^{[2]} = dz_1^{[2]} = -0.526.$$

new weights

$$\begin{aligned} w_{11}^{[2]}_{\text{new}} &= w_{11}^{[2]}_{\text{old}} - \alpha \cdot dw_{11}^{[2]} \\ &= -0.3 - 0.9 \times (-0.174106) \\ &= \boxed{-0.1433046} \end{aligned}$$

$$\begin{aligned} w_{12}^{[2]}_{\text{new}} &= -0.2 - 0.9 \times (-0.27615) \\ &= -0.2 + 0.2485 \\ &= \underline{\underline{-0.9515}} \end{aligned}$$

$$\begin{aligned} w_{12}^{[2]}_{\text{new}} &= w_{12}^{[2]}_{\text{old}} - \alpha \cdot dw_{12}^{[2]} \\ &= -0.2 - 0.9 \times (-0.27615) \\ &= -0.2 + 0.2485 \\ &= \boxed{0.0485} \end{aligned}$$

$$\begin{aligned} b_1^{[2]} &= 0.1 - 0.9 \times (-0.526) \\ &= \boxed{0.5734} \end{aligned}$$

if mse cost funⁿ

$$dz_1^{[2]} = a_1^{[2]} \times (1 - a_1^{[2]}) \times (y - a_1^{[2]})$$

$L1-N1$

$$\begin{aligned} (1) \quad dz_1^{[1]} &= (dz_1^{[2]} \cdot w_{11}^{[2]}) * g'(z_1^{[1]}) \\ &= (-0.526 * (-0.3)) * a_1^{[1]} (1 - a_1^{[1]}) \\ &= 0.1578 * 0.331 (1 - 0.331) \\ &= 0.0349 \end{aligned}$$

$$\begin{aligned} (2) \quad dw_{11}^{[1]} &= dz_1^{[1]} \cdot x_1 \\ &= 0.0349 * 1 = 0.0349 \end{aligned}$$

$$\begin{aligned} (3) \quad dw_{12}^{[1]} &= dz_1^{[1]} \cdot x_2 \\ &= 0.0349 * 0 = 0 \end{aligned}$$

$$(3) \quad dw_{13}^{[1]} = dz_1^{[1]} \cdot x_3 = 0.0349$$

$$(4) \quad db_1^{[1]} = dz_1^{[1]} = 0.0349$$

weight updates.

$$\begin{aligned} (1) \quad w_{11}^{[1]}_{\text{new}} &= w_{11}^{[1]}_{\text{old}} - 0.9 * dw_{11}^{[1]} \\ &= 0.2 - 0.9 * 0.0349 \\ &= 0.2 - 0.03141 = \boxed{0.16859} \end{aligned}$$

$$w_{12}^{[1]}_{\text{new}} = 0.4 - 0.9 * 0 = \boxed{0.4}$$

$$\begin{aligned} w_{13}^{[1]}_{\text{new}} &= w_{13}^{[1]}_{\text{old}} - \alpha * dw_{13}^{[1]} \\ &= -0.5 - 0.9 * 0.0349 \\ &= \boxed{-0.53141} \end{aligned}$$

$$b_1^{[1]} = -0.4 - 0.9 * 0.0349 = -0.43141$$

innovate

achieve

lead

$x_1 - N_2$

$$\begin{aligned} dz_2^{(1)} &= dz_1^{(2)} \cdot w_{12}^{(2)} \cdot g'(z_2^{(1)}) \\ &= (-0.526 \times -0.2) \times a_2^{(0)} (1 - a_2^{(1)}) \\ &= 0.1052 \times 0.525 (1 - 0.525) \\ &= \boxed{0.0262} \end{aligned}$$

$$\begin{aligned} \textcircled{2} \quad dw_{21}^{(1)} &= dz_2^{(1)} \cdot x_1 \\ &= \boxed{0.0262} \end{aligned}$$

$$\textcircled{3} \quad dw_{22}^{(1)} = dz_2^{(1)} \cdot x_2 = \boxed{0}$$

$$\textcircled{3} \quad dw_{23}^{(1)} = dz_2^{(1)} \cdot x_3 = \boxed{0.0262}$$

$$\textcircled{4} \quad db_2^{(1)} = dz_2^{(1)} = \boxed{0.0262}$$

weight updates

$$\begin{aligned} w_{21}^{(1)}_{\text{new}} &= w_{21}^{(1)}_{\text{old}} - \alpha \cdot dw_{21}^{(1)} \\ &= -0.3 - 0.9 \times 0.0262 = \boxed{-0.32358} \end{aligned}$$

$$\begin{aligned} w_{22}^{(1)}_{\text{new}} &= w_{22}^{(1)}_{\text{old}} - \alpha \cdot dw_{22}^{(1)} \\ &= 0.1 - 0.9 \times 0 = \boxed{0.1} \end{aligned}$$

$$\begin{aligned} w_{23}^{(1)}_{\text{new}} &= w_{23}^{(1)}_{\text{old}} - \alpha \cdot dw_{23}^{(1)} \\ &= 0.2 - 0.9 \times 0.0262 \\ &= \boxed{0.17642} \end{aligned}$$

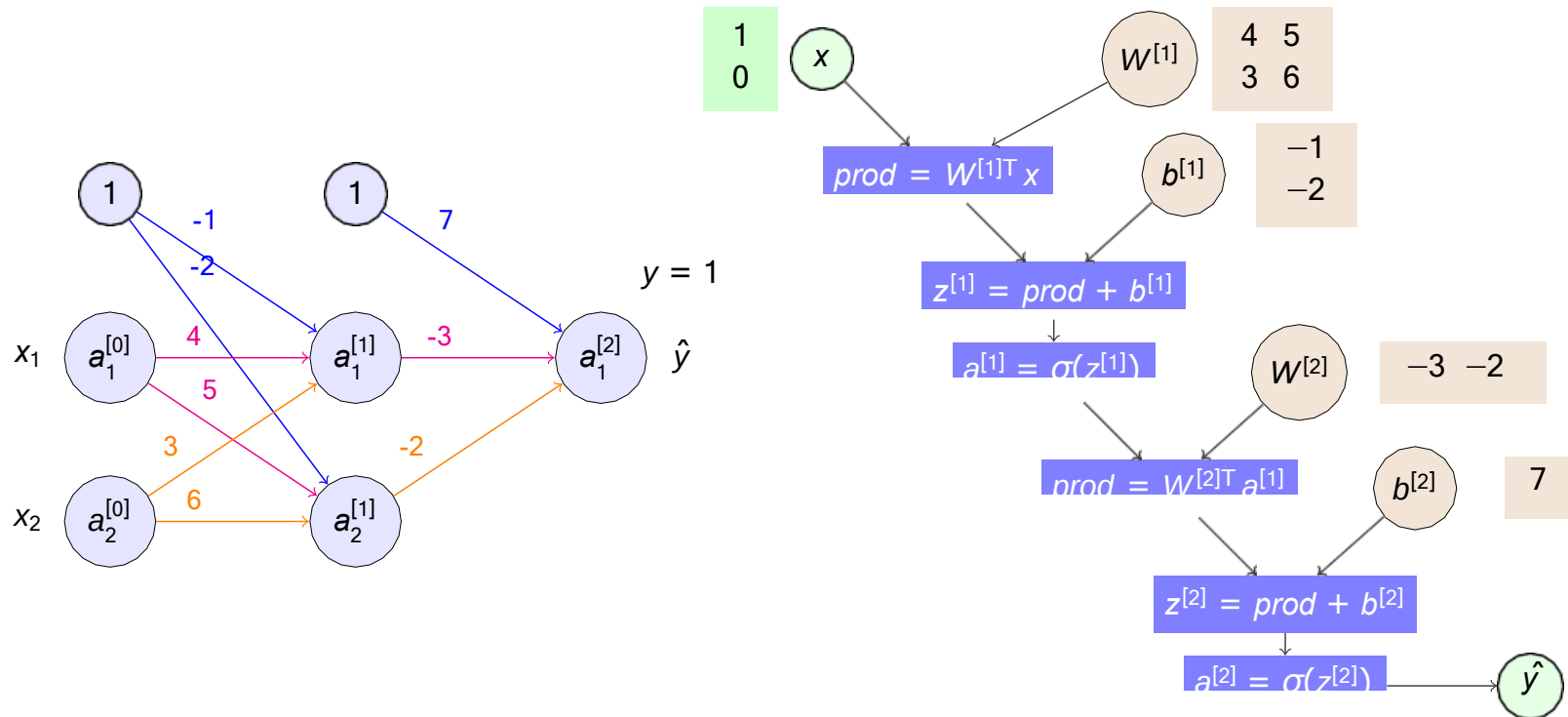
$$b_2^{(1)} = 0.2 - 0.9 \times 0.0262 = \boxed{0.17642}$$

innovate

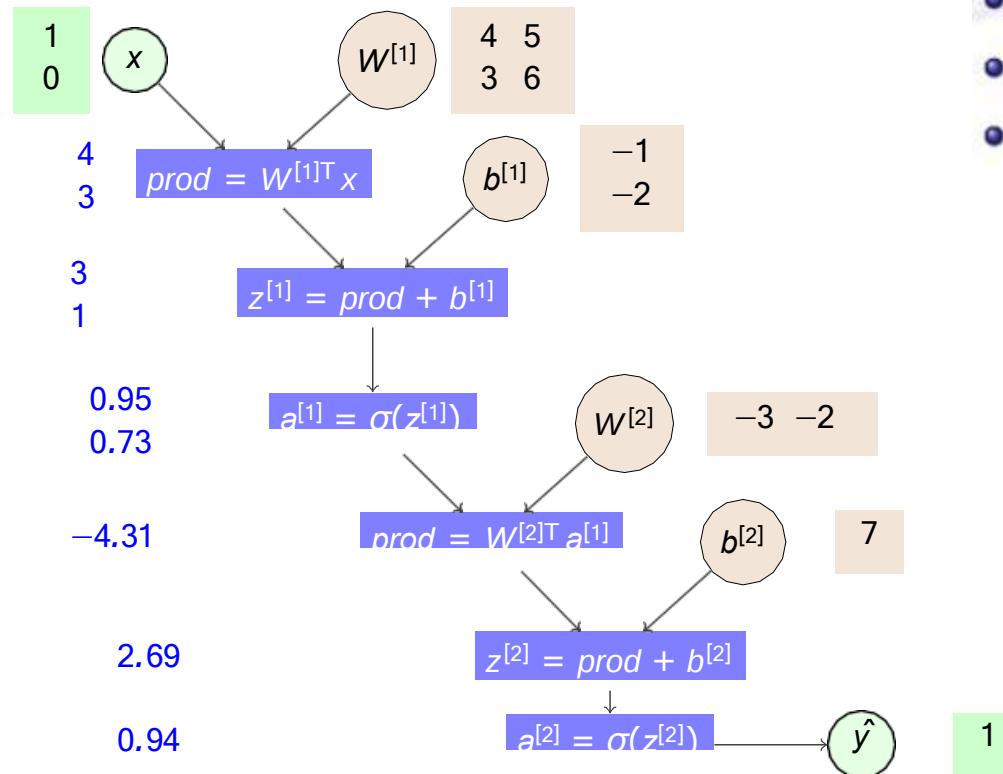
achieve

lead

Exercise – With vectorization

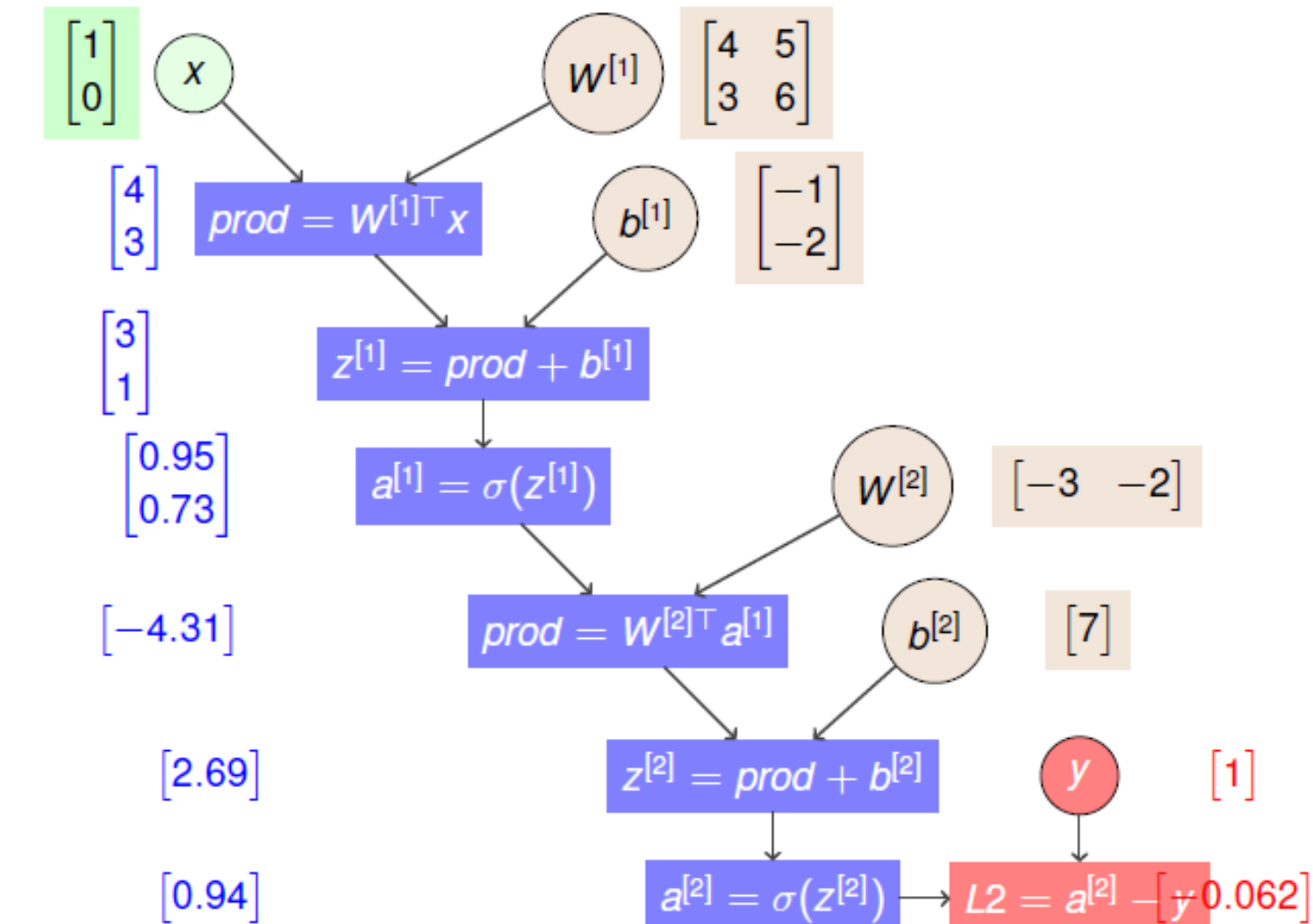


Computation Graph for Forward Pass

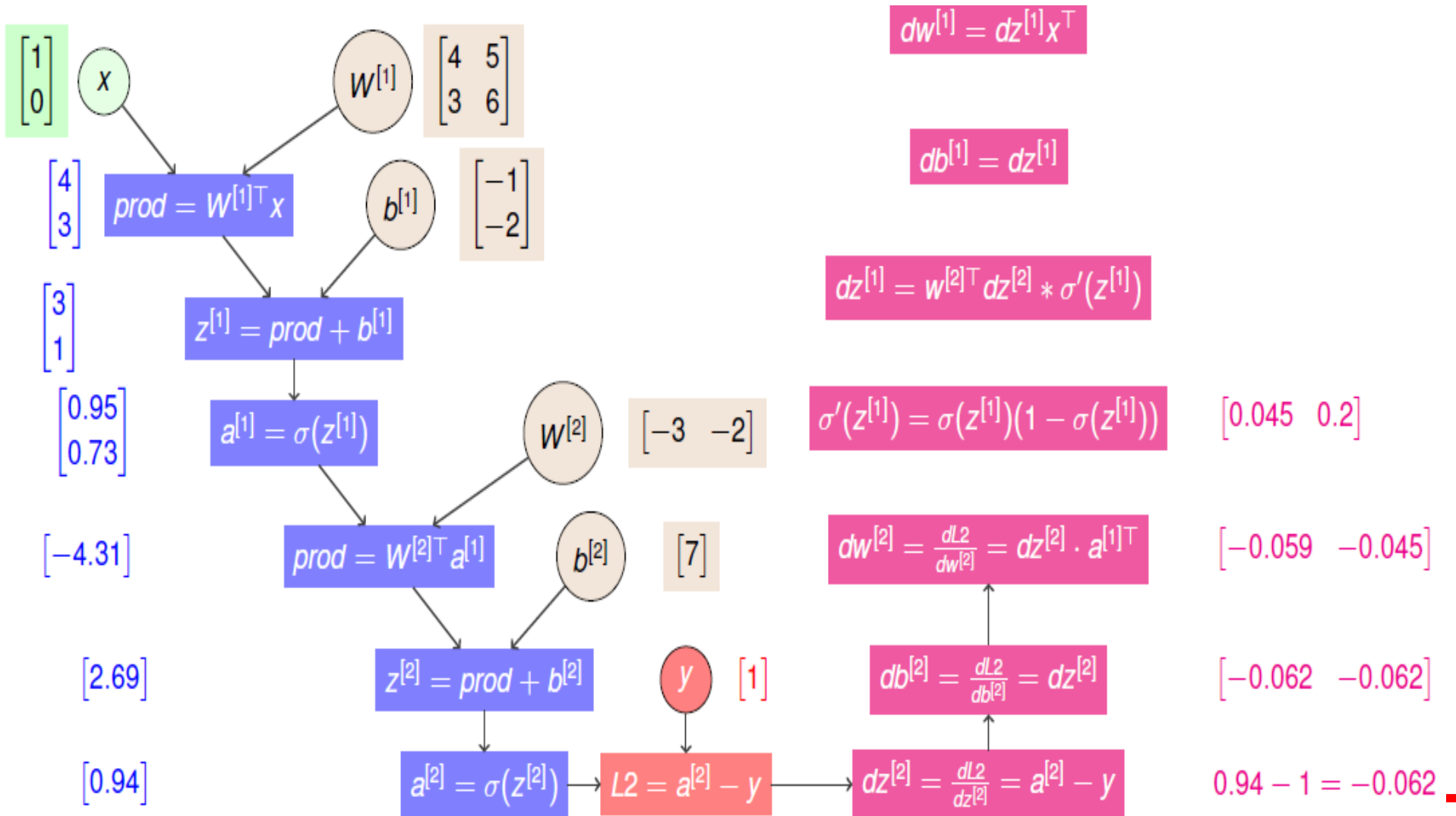


- $W^{[l]} : n^{[l]} \times n^{[l-1]}$
- $b^{[l]} : n^{[l]} \times 1$
- $z^{[l]} : n^{[l]} \times 1$
- $a^{[l]} : n^{[l]} \times 1$
- $dW^{[l]} : n^{[l]} \times n^{[l-1]}$
- $db^{[l]} : n^{[l]} \times 1$
- $dz^{[l]} : n^{[l]} \times 1$
- $da^{[l]} : n^{[l]} \times 1$

Computation Graph for Cost Function



Computation Graph for Backward Pass



Demo



<https://playground.tensorflow.org/>

Practice problems

Example 1 Computational Graph for Back Propagation



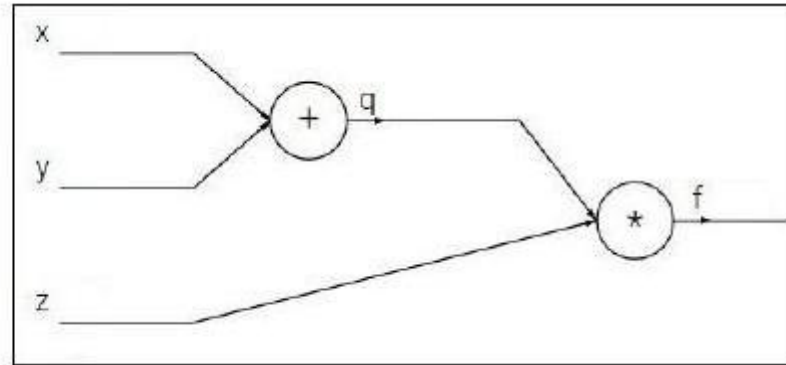
$$f(x, y, z) = (x + y)z$$



Computational Graph for Back Propagation



$$f(x, y, z) = (x + y)z$$

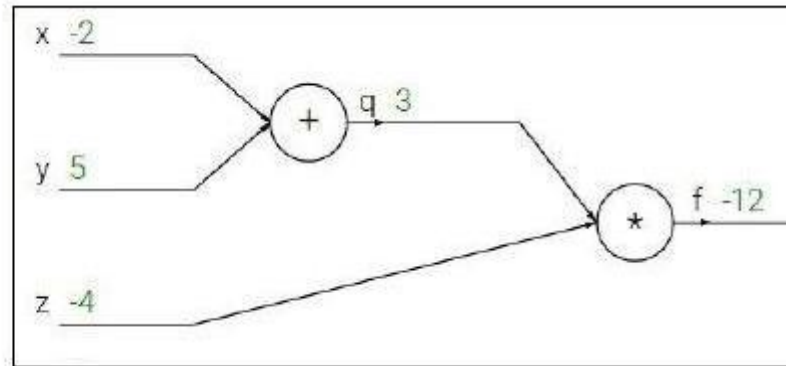


Computational Graph for Back Propagation



$$f(x, y, z) = (x + y)z$$

e.g. $x = -2$, $y = 5$, $z = -4$

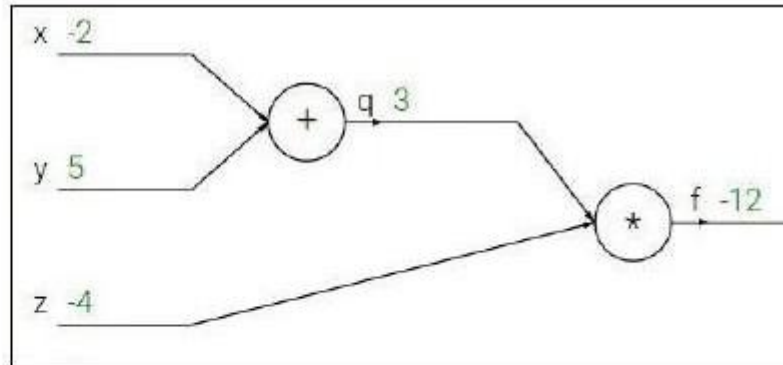


Computational Graph for Back Propagation



$$f(x, y, z) = (x + y)z$$

e.g. $x = -2, y = 5, z = -4$



Want: $\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}, \frac{\partial f}{\partial z}$

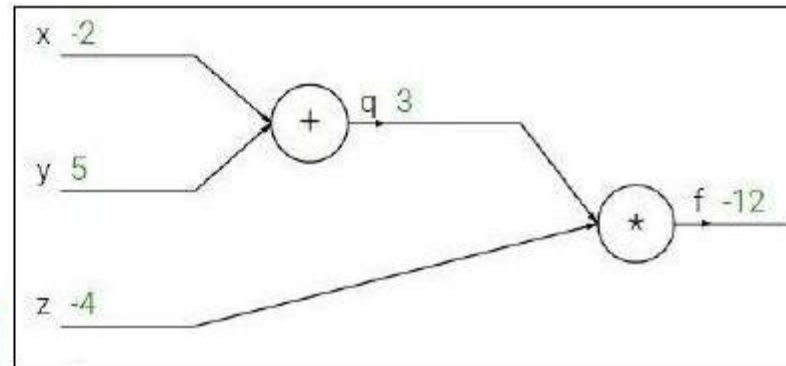
Computational Graph for Back Propagation



$$f(x, y, z) = (x + y)z$$

e.g. $x = -2, y = 5, z = -4$

$$q = x + y \quad \frac{\partial q}{\partial x} = 1, \frac{\partial q}{\partial y} = 1$$



Want: $\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}, \frac{\partial f}{\partial z}$

Computational Graph for Back Propagation

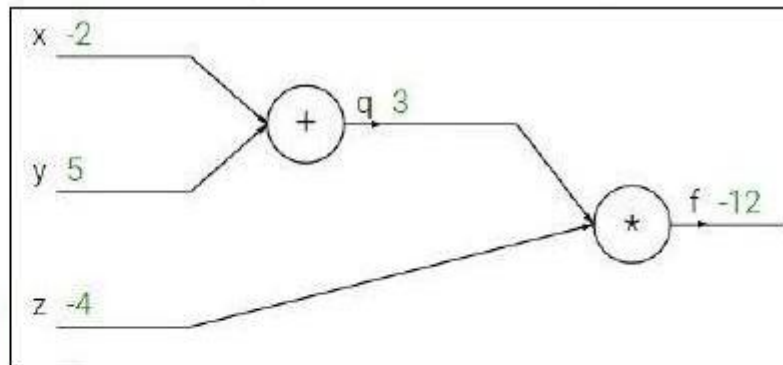
$$f(x, y, z) = (x + y)z$$

e.g. $x = -2, y = 5, z = -4$

$$q = x + y \quad \frac{\partial q}{\partial x} = 1, \frac{\partial q}{\partial y} = 1$$

$$f = qz \quad \frac{\partial f}{\partial q} = z, \frac{\partial f}{\partial z} = q$$

Want: $\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}, \frac{\partial f}{\partial z}$



Computational Graph for Back Propagation



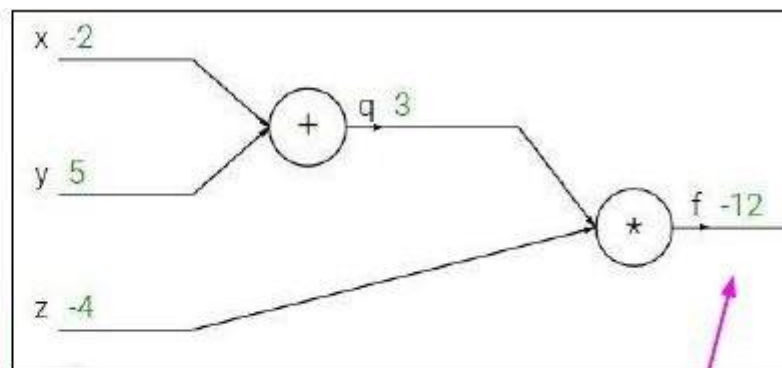
$$f(x, y, z) = (x + y)z$$

e.g. $x = -2, y = 5, z = -4$

$$q = x + y \quad \frac{\partial q}{\partial x} = 1, \frac{\partial q}{\partial y} = 1$$

$$f = qz \quad \frac{\partial f}{\partial q} = z, \frac{\partial f}{\partial z} = q$$

Want: $\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}, \frac{\partial f}{\partial z}$



$$\frac{\partial f}{\partial f}$$

Computational Graph for Back Propagation



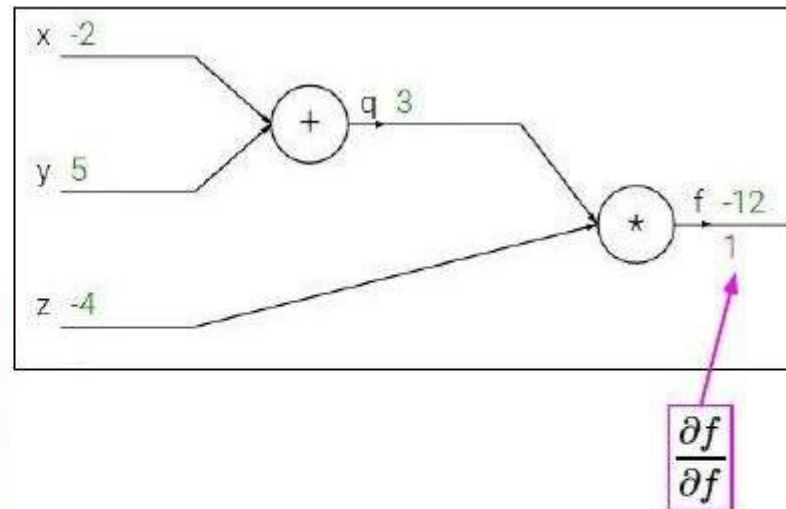
$$f(x, y, z) = (x + y)z$$

e.g. $x = -2, y = 5, z = -4$

$$q = x + y \quad \frac{\partial q}{\partial x} = 1, \frac{\partial q}{\partial y} = 1$$

$$f = qz \quad \frac{\partial f}{\partial q} = z, \frac{\partial f}{\partial z} = q$$

Want: $\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}, \frac{\partial f}{\partial z}$



Computational Graph for Back Propagation



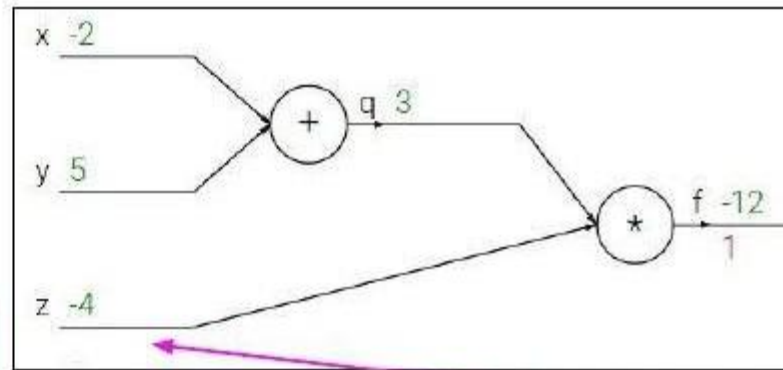
$$f(x, y, z) = (x + y)z$$

e.g. $x = -2, y = 5, z = -4$

$$q = x + y \quad \frac{\partial q}{\partial x} = 1, \frac{\partial q}{\partial y} = 1$$

$$f = qz \quad \frac{\partial f}{\partial q} = z, \frac{\partial f}{\partial z} = q$$

Want: $\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}, \frac{\partial f}{\partial z}$



$$\frac{\partial f}{\partial z}$$

Computational Graph for Back Propagation



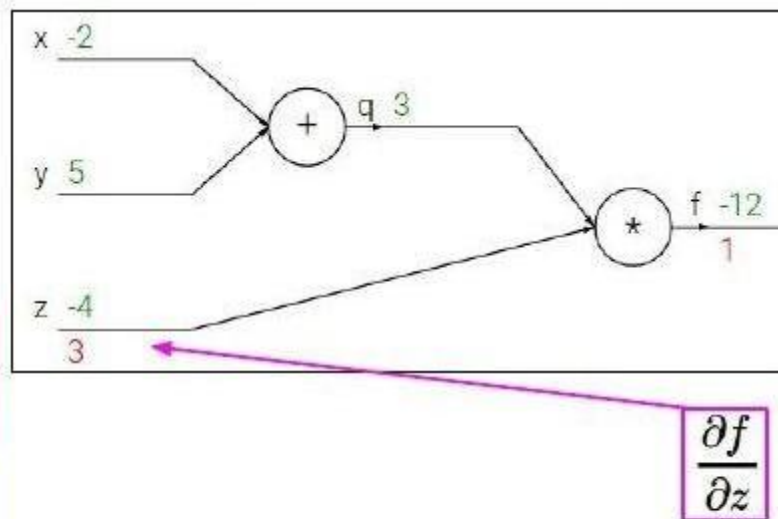
$$f(x, y, z) = (x + y)z$$

e.g. $x = -2, y = 5, z = -4$

$$q = x + y \quad \frac{\partial q}{\partial x} = 1, \frac{\partial q}{\partial y} = 1$$

$$f = qz \quad \frac{\partial f}{\partial q} = z, \frac{\partial f}{\partial z} = q$$

Want: $\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}, \frac{\partial f}{\partial z}$



Computational Graph for Back Propagation



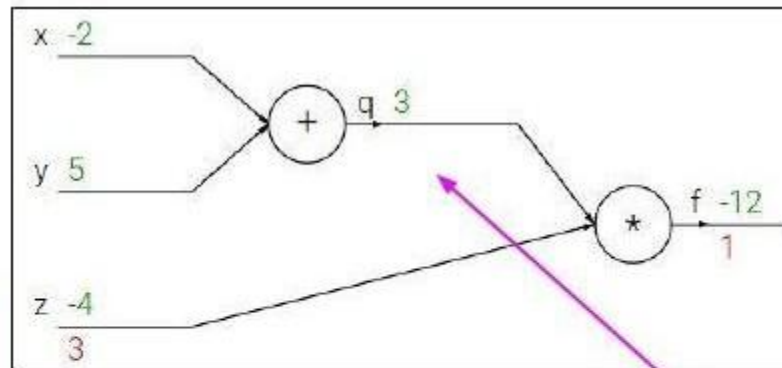
$$f(x, y, z) = (x + y)z$$

e.g. $x = -2$, $y = 5$, $z = -4$

$$q = x + y \quad \frac{\partial q}{\partial x} = 1, \frac{\partial q}{\partial y} = 1$$

$$f = qz \quad \frac{\partial f}{\partial q} = z, \frac{\partial f}{\partial z} = q$$

Want: $\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}, \frac{\partial f}{\partial z}$



$$\frac{\partial f}{\partial q}$$

Computational Graph for Back Propagation

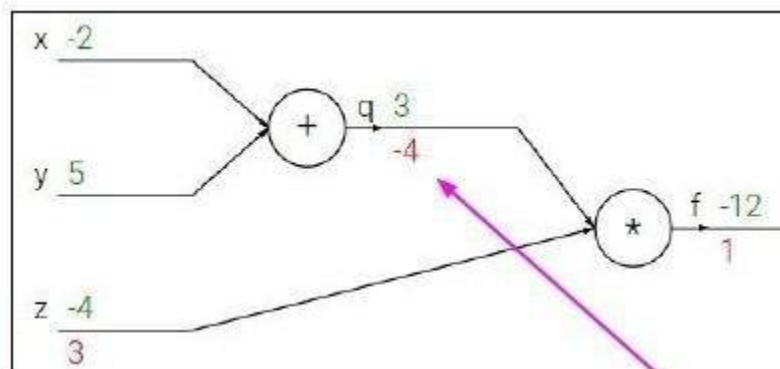
$$f(x, y, z) = (x + y)z$$

e.g. $x = -2, y = 5, z = -4$

$$q = x + y \quad \frac{\partial q}{\partial x} = 1, \frac{\partial q}{\partial y} = 1$$

$$f = qz \quad \frac{\partial f}{\partial q} = z, \frac{\partial f}{\partial z} = q$$

Want: $\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}, \frac{\partial f}{\partial z}$



$$\frac{\partial f}{\partial q}$$

Computational Graph for Back Propagation

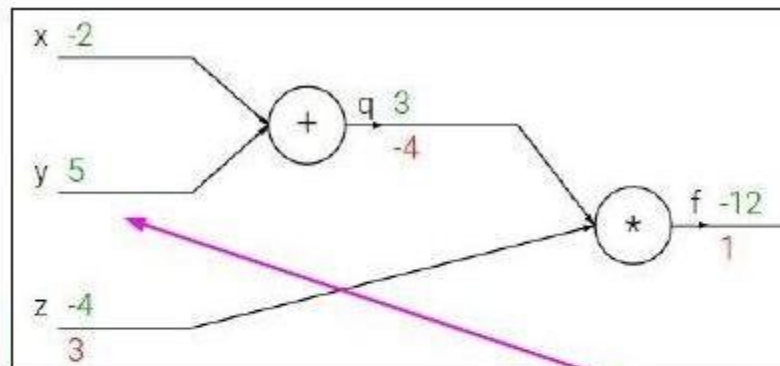
$$f(x, y, z) = (x + y)z$$

e.g. $x = -2, y = 5, z = -4$

$$q = x + y \quad \frac{\partial q}{\partial x} = 1, \frac{\partial q}{\partial y} = 1$$

$$f = qz \quad \frac{\partial f}{\partial q} = z, \frac{\partial f}{\partial z} = q$$

Want: $\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}, \frac{\partial f}{\partial z}$



$$\frac{\partial f}{\partial y}$$

Chain rule:

$$\frac{\partial f}{\partial y} = \frac{\partial f}{\partial q} \frac{\partial q}{\partial y}$$

Upstream
gradient

Local
gradient

Computational Graph for Back Propagation



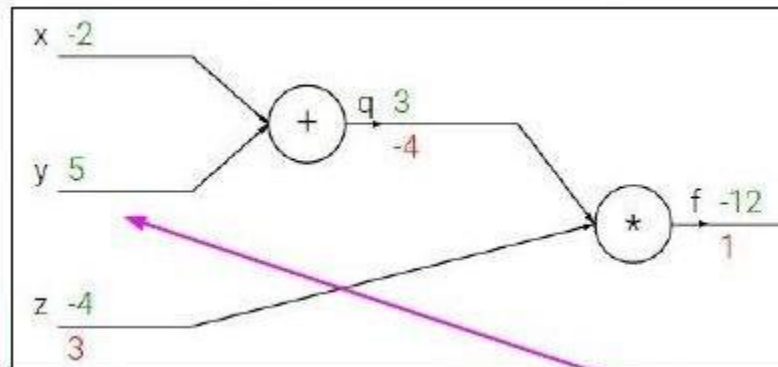
$$f(x, y, z) = (x + y)z$$

e.g. $x = -2, y = 5, z = -4$

$$q = x + y \quad \frac{\partial q}{\partial x} = 1, \frac{\partial q}{\partial y} = 1$$

$$f = qz \quad \frac{\partial f}{\partial q} = z, \frac{\partial f}{\partial z} = q$$

Want: $\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}, \frac{\partial f}{\partial z}$



$$\frac{\partial f}{\partial y}$$

Chain rule:

$$\frac{\partial f}{\partial y} = \frac{\partial f}{\partial q} \frac{\partial q}{\partial y}$$

Upstream
gradient

Local
gradient

Computational Graph for Back Propagation



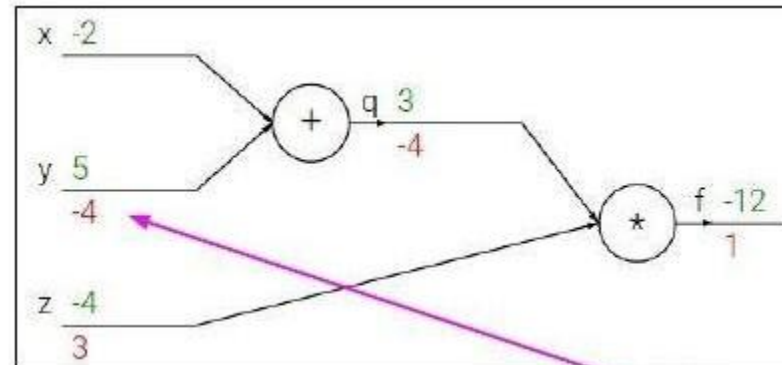
$$f(x, y, z) = (x + y)z$$

e.g. $x = -2, y = 5, z = -4$

$$q = x + y \quad \frac{\partial q}{\partial x} = 1, \frac{\partial q}{\partial y} = 1$$

$$f = qz \quad \frac{\partial f}{\partial q} = z, \frac{\partial f}{\partial z} = q$$

Want: $\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}, \frac{\partial f}{\partial z}$



Chain rule:

$$\frac{\partial f}{\partial y} = \frac{\partial f}{\partial q} \frac{\partial q}{\partial y}$$

Upstream
gradient

Local
gradient

$$\frac{\partial f}{\partial y}$$

Computational Graph for Back Propagation



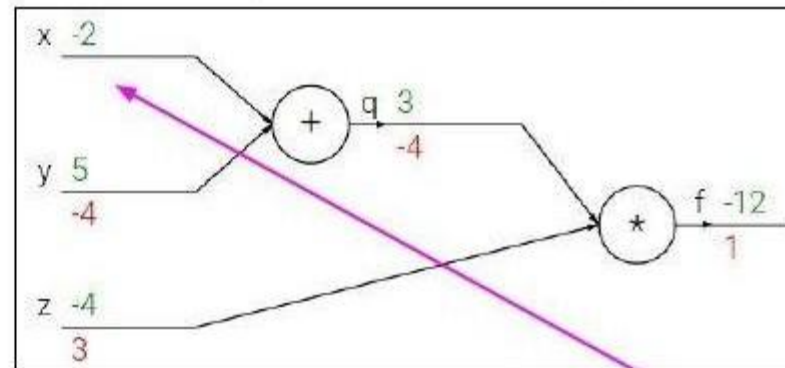
$$f(x, y, z) = (x + y)z$$

e.g. $x = -2, y = 5, z = -4$

$$q = x + y \quad \frac{\partial q}{\partial x} = 1, \frac{\partial q}{\partial y} = 1$$

$$f = qz \quad \frac{\partial f}{\partial q} = z, \frac{\partial f}{\partial z} = q$$

Want: $\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}, \frac{\partial f}{\partial z}$



$$\frac{\partial f}{\partial x}$$

Chain rule:

$$\frac{\partial f}{\partial x} = \frac{\partial f}{\partial q} \frac{\partial q}{\partial x}$$

Upstream
gradient

Local
gradient

Computational Graph for Back Propagation



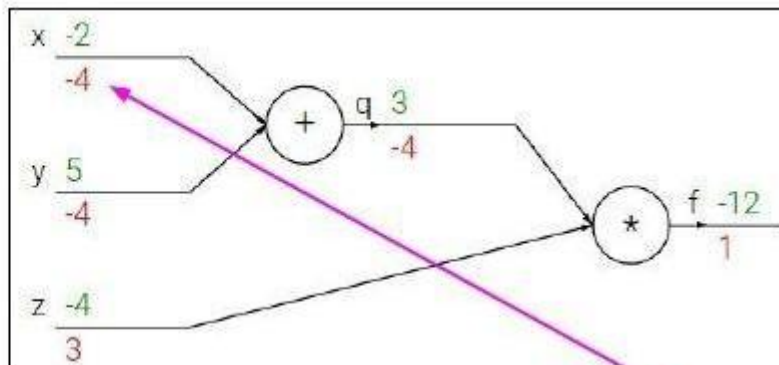
$$f(x, y, z) = (x + y)z$$

e.g. $x = -2, y = 5, z = -4$

$$q = x + y \quad \frac{\partial q}{\partial x} = 1, \frac{\partial q}{\partial y} = 1$$

$$f = qz \quad \frac{\partial f}{\partial q} = z, \frac{\partial f}{\partial z} = q$$

Want: $\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}, \frac{\partial f}{\partial z}$



$$\frac{\partial f}{\partial x}$$

Chain rule:

$$\frac{\partial f}{\partial x} = \frac{\partial f}{\partial q} \frac{\partial q}{\partial x}$$

Upstream
gradient

Local
gradient

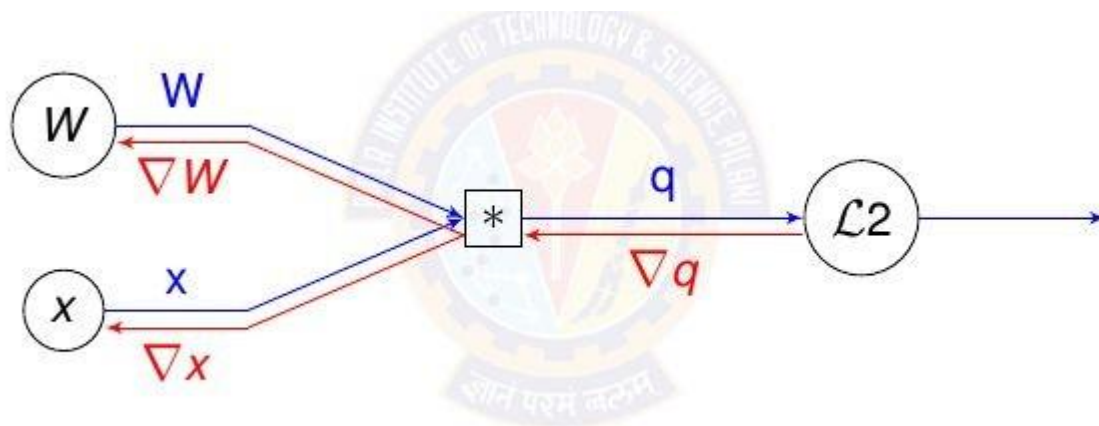
Example 2

Draw the computational graph for the equation

$$f(x, w) = || W \cdot x ||^2 = \sum_{i=1}^n (W \cdot x)_i^2$$

Using the computation graph show the computation of the gradients also. Assume that

$$W = \begin{bmatrix} 0.1 & 0.5 \\ -0.3 & 0.8 \end{bmatrix}$$
$$x = \begin{bmatrix} 0.2 \\ 0.4 \end{bmatrix}$$



$$W = \begin{bmatrix} 0.1 & 0.5 \\ -0.3 & 0.8 \end{bmatrix}$$

$$x = \begin{bmatrix} 0.2 \\ 0.4 \end{bmatrix}$$

$$q = W \cdot x = \begin{bmatrix} 0.1 * 0.2 + 0.5 * 0.4 \\ -0.3 * 0.2 + 0.8 * 0.4 \end{bmatrix} = \begin{bmatrix} 0.22 \\ 0.26 \end{bmatrix}$$

$$L2 = \sum_{i=1} q_i^2 = 0.22^2 + 0.26^2 = 0.116$$

$$\nabla q = \nabla_q f = 2q$$

$$= 2 \begin{bmatrix} 0.22 \\ 0.26 \end{bmatrix} = \begin{bmatrix} 0.44 \\ 0.52 \end{bmatrix}$$

$$\nabla W = \nabla_W f = 2q \cdot x^\top$$

$$= 2 \begin{bmatrix} 0.22 \\ 0.26 \end{bmatrix} \begin{bmatrix} 0.2 & 0.4 \end{bmatrix} = \begin{bmatrix} 0.088 & 0.176 \\ 0.105 & 0.208 \end{bmatrix}$$

$$\nabla x = \nabla_x f = 2W^\top \cdot q$$

$$= 2 \begin{bmatrix} 0.1 & -0.3 \\ 0.5 & 0.8 \end{bmatrix} \begin{bmatrix} 0.22 \\ 0.26 \end{bmatrix} = \begin{bmatrix} -0.112 \\ 0.636 \end{bmatrix}$$

Optional

Derivative of cost function w.r.t final layer linear function



$$\frac{\partial C}{\partial z^{[L]}} = \frac{\partial C}{\partial a^{[L]}} \cdot \frac{\partial a^{[L]}}{\partial z^{[L]}} = \frac{\partial C}{\partial a^{[L]}} \cdot * \sigma'(z^{[L]})$$

$$\boxed{\frac{\partial C}{\partial z^{[L]}} = \frac{\partial C}{\partial a^{[L]}} \cdot * \sigma'(z^{[L]})}$$

where,

$$\frac{\partial C}{\partial a^{[L]}} = \frac{\partial(-(y \log a^{[L]} + (1 - y) \log(1 - a^{[L]})))}{\partial a^{[L]}}$$

using

$$\frac{\partial y \log a^{[L]}}{\partial a^{[L]}} = \frac{y}{a^{[L]}} \quad \text{and} \quad \frac{\partial (1 - y) \log(1 - a^{[L]})}{\partial a^{[L]}} = \frac{1 - y}{1 - a^{[L]}} (-1)$$

we get:

$$\frac{\partial(-(y \log a^{[L]} + (1 - y) \log(1 - a^{[L]})))}{\partial a^{[L]}} = -\left(\frac{y}{a^{[L]}} - \frac{1 - y}{1 - a^{[L]}}\right)$$

$$\therefore \frac{\partial C}{\partial a^{[L]}} = -\left(\frac{y}{a^{[L]}} - \frac{1 - y}{1 - a^{[L]}}\right) \dots 1$$

linear function

(Derivative of sigmoid activation function)



$$\begin{aligned}\sigma(z^{[L]}) &= \frac{1}{1 + e^{-z^{[L]}}} \\ \sigma'(z^{[L]}) &= \frac{d}{dz^{[L]}} \left(\frac{1}{1 + e^{-z^{[L]}}} \right) \\ &= -\frac{1}{(1 + e^{-z^{[L]}})^2} \cdot \frac{d}{dz^{[L]}} (1 + e^{-z^{[L]}}) \\ &= -\frac{e^{-z^{[L]}}}{(1 + e^{-z^{[L]}})^2} \cdot \frac{d}{dz^{[L]}} (-z^{[L]}) \\ &= \frac{e^{-z^{[L]}}}{(1 + e^{-z^{[L]}})^2} \\ &= \frac{e^{-z^{[L]}} + 1 - 1}{(1 + e^{-z^{[L]}})^2} \\ &= \frac{1}{1 + e^{-z^{[L]}}} \cdot \left(\frac{e^{-z^{[L]}} + 1 - 1}{1 + e^{-z^{[L]}}} \right) \\ &= \frac{1}{1 + e^{-z^{[L]}}} \cdot \left(\frac{1 + e^{-z^{[L]}}}{1 + e^{-z^{[L]}}} - \frac{1}{1 + e^{-z^{[L]}}} \right) \\ &= \frac{1}{1 + e^{-z^{[L]}}} \cdot \left(1 - \frac{1}{1 + e^{-z^{[L]}}} \right) \\ &= \sigma(z^{[L]}) \cdot (1 - \sigma(z^{[L]})) \\ \sigma'(z^{[L]}) &= \sigma(z^{[L]}) \cdot (1 - \sigma(z^{[L]})) \\ \therefore \sigma'(z^{[L]}) &= a^{[L]} \cdot (1 - a^{[L]}) \quad \dots 2\end{aligned}$$

Derivative of cost function w.r.t final layer linear function

innovate

achieve

lead

From equation 1, 2:

$$\begin{aligned}\frac{\partial C}{\partial z^{[L]}} &= -\left(\frac{y}{a^{[L]}} - \frac{1-y}{1-a^{[L]}}\right) \cdot a^{[L]} \cdot (1-a^{[L]}) \\&= -\left(\frac{y(1-a^{[L]}) - (1-y)a^{[L]}}{a^{[L]} \cdot (1-a^{[L]})}\right) \cdot a^{[L]} \cdot (1-a^{[L]}) \\&= -(y(1-a^{[L]}) - (1-y)a^{[L]}) \\&= -(y - ya^{[L]} - (a^{[L]} - ya^{[L]})) \\&= -(y - ya^{[L]} - a^{[L]} + ya^{[L]}) \\&= -(y - a^{[L]}) \\&= a^{[L]} - y\end{aligned}$$

$$\therefore \boxed{\frac{\partial C}{\partial z^{[L]}} = a^{[L]} - y}$$



Thank You All !